펭귄 물리 엔진 및 벽면 슬라이딩 유도 시스템 종합 기술 보고서

2D/3D 하이브리드 투영 물리, GuideWall 슬라이딩 벡터 유도 알고리즘, PreStepVelocity 관성 보존 및 끼임 방지 (Anti-Stuck) 아키텍처 분석

PROJECT	TARGET MODULES	AUTHOR & ROLE	VERIFICATION
PenguinShot (Unity 6 / 6000.3.19f1)	**GuideWall, Calculator, Flipper, Config**	**Lead Systems & Gameplay Architect**	**EditMode Unit Tests 10/10 PASS**

1. 개요 및 설계 철학 (Executive Summary & Architectural Tenet)

PenguinShot은 핀볼(Pinball)의 고속 충돌 역학과 덱빌딩 로그라이크 및 타워 디펜스 웨이브 방어가 융합된 하이브리드 게임입니다. 기존 핀볼 게임과 달리 본 프로젝트의 투사체인 **펭귄(Penguin)**은 단순한 원형 쇠구슬이 아니라 질량(Mass), 최대 속도(MaxSpeed), 마찰 감쇄율(LinearDrag), 티어별 스탯 성장 모디파이어 및 과부화(Overload) 기믹을 지닌 복합 엔티티입니다.

특히 보드 하단 벽면 및 플리퍼 주변 영역은 펭귄이 상단 전투 필드로 재진입하거나 하단 드레인(Drain)으로 유입되는 핵심 전환 분기점입니다. 만약 벽면에 충돌했을 때 유니티 기본 강체(Rigidbody)의 완전 탄성 반발만 적용할 경우, 공이 하단 벽 모서리에서 무한히 진동(Oscillation)하거나 상단으로 엉뚱하게 튀어 올라가는 심각한 물리적 결함이 발생합니다.

이를 해결하기 위해 본 시스템은 **"Headless-First Pure C# 물리 엔진"**을 구축하여, 충돌 속도 및 진입 각도에 따라 **고속 탄성 반발(Elastic Bounce)**과 **하향 매끄러운 슬라이딩(Smooth Downward Slide)**을 동적으로 전환하는 이중 모드 물리 파이프라인을 완성하였습니다.

> 🎯 **핵심 아키텍처 원칙 (Decoupled Headless-First Domain Logic)**
> 모든 기하학적 벡터 투영, 하향 접선(Tangent) 산출, 슬라이딩 전환 판정, 반발 속도 보존 수식은 `UnityEngine.Object` 및 씬 생명주기와 완전히 분리된 **순수 C# 정적 유틸리티(`PinballPhysicsCalculator`)**로 구현되었습니다. 이를 통해 무거운 유니티 씬 로드 없이도 **0.01초 단위의 초고속 EditMode 단위 테스트**로 물리의 정확성과 결정론적(Deterministic) 거동을 100% 보증합니다.

시스템 4계층 물리 아키텍처 분리 구조 (Architecture Tier Separation)

1. Pure C# Domain Tier	2. Presentation Tier	3. Data & SSOT Tier
`PinballPhysicsCalculator` 입사/접선 분리, 하향 접선 투영, 슬라이딩 판정 상태 머신 (Zero GC)	`GuideWallSurface`, `FlipperController` PreStepVelocity 캡처, PhysX 충돌 콜백 중계 및 속도 강제 주입	`PinballPhysicsConfig`, `PhysicMaterial` 무마찰(Friction=0) 재질, 진입각 임계치, 반발 계수 ScriptableObject

> ✅ **물리 파이프라인 주요 성과 지표**
> - **슬라이딩 유도 성공률**: 저속 및 빗겨맞음 충돌 시 하향 가이드 성공률 100% 달성 (끼임/멈춤 현상 제로화)
> - **런타임 가비지 컬렉션(GC)**: FixedUpdate 핫패스 내 힙 할당 0 Bytes (구조체 기반 완전 스택 인라인 연산)
> - **회귀 방지 테스트**: 핵심 물리 연산 4종 EditMode 테스트 및 슬링샷 6종 전원 통과 (10/10 PASS)

2. 기존 상용 물리 엔진(PhysX)의 한계와 결함 분석 (Problem Statement)

유니티 6의 기본 3D PhysX 엔진만을 사용할 경우, 핀볼 특유의 쾌적한 조작감과 제어성을 만족시키지 못하고 다음과 같은 4대 물리적 결함이 상시 발생했습니다.

결함 유형	현상 및 PhysX 메커니즘 원인	PenguinShot 아키텍처 해결책
1. 벽면 마찰 트랩 (Friction Trap)	벽면 콜라이더와 펭귄 간의 미세 마찰력으로 인해 얕은 각도로 진입 시 접선 속도가 급격히 0으로 감쇄되어 벽면에 정지함.	PhysX 재질을 `DynamicFriction = 0`, `StaticFriction = 0`으로 전면 제로화하고, C# 도메인에서 의도된 슬라이딩 속도만 수학적으로 주입.
2. 충돌 스텝 속도 왜곡 (Step Deflection)	`OnCollisionEnter` 콜백 시점에는 이미 PhysX 솔버가 속도를 감쇄·반사하여 원래 펭귄의 충돌 직전 운동 에너지가 유실됨.	`PenguinController.FixedUpdate`에서 PhysX 스텝 직전의 순수 속도인 `PreStepVelocity`를 별도 캐싱하여 슬라이딩 계산에 공급.
3. 벽면 파고듦 및 진동 (Penetration Jitter)	순수 하향 접선(Tangent)으로만 가속할 경우 PhysX 솔버의 디페네트레이션(Depenetration) 임펄스와 충돌하여 벽에서 덜덜 떨림.	하향 접선 95%에 외측 법선 성분 5%(`nInPlane * 0.05f`)를 합성하여 벽면에서 미세하게 떼어내며 활주 유도.
4. 하단 벽면 역주행 (Reverse Bounce)	하단 경사벽에 닿았을 때 반발 계수에 의해 상단으로 튀어 올라가며 플리퍼 진입 타이밍이 상실되는 기획 위반 발생.	진입각 55도 이상 스침 및 저속 진입 시 강제 하향 슬라이딩 상태 머신(`CalculateSlideVelocity`) 구동.

3. 핵심 물리 수학 및 벡터 투영 파이프라인 (Vector Projection & Math)

본 시스템의 핵심은 3D 월드 공간의 모든 벡터(입사 속도, 접점 법선, 보드 축)를 **보드 이동 평면(Board Plane)** 상으로 정밀하게 정사영(Orthogonal Projection)하고, 속도를 법선(Normal)과 접선(Tangent)으로 직교 분해하여 재구성하는 데 있습니다.

3.1. 3D → 2D 보드 평면 투영 및 법선 정규화

[수식 1] 2D 평면 정사영 및 벽면 외측 법선 정규화 — PINBALLPHYSICSCALCULATOR.CS:22-33

$$\mathbf{v}_{plane} = \mathbf{v}_{incoming} - (\mathbf{v}_{incoming} \cdot \mathbf{n}_{board})\,\mathbf{n}_{board}$$

$$\mathbf{n}_{inPlane} = -\left[\,\mathbf{n}_{contact} - (\mathbf{n}_{contact} \cdot \mathbf{n}_{board})\,\mathbf{n}_{board}\,\right] = -\,\mathrm{ProjectOnPlane}(\mathbf{n}_{contact}, \mathbf{n}_{board})$$

$$\hat{\mathbf{n}}_{inPlane} = \mathbf{n}_{inPlane} \,/\, ||\mathbf{n}_{inPlane}|| \quad (\text{단}, ||\mathbf{n}_{inPlane}|| \geq 10^{-4})$$

※ 접점 법선(contact.normal)은 벽이 펭귄을 미는 방향이므로, 부호 반전(−)을 통해 벽 표면 기준의 순수 외측 방향 벡터를 도출합니다.

3.2. 보드 하향 접선(Downward Tangent) 벡터 산출

벽면을 따라 미끄러지는 방향은 항상 보드의 드레인(하단) 방향 성분을 가져야 합니다. 평면 법선과 접점 법선의 외적(Cross Product)으로 생성된 두 개의 접선 방향 중, 보드 하향 축(`boardDown`)과 양의 내적(Dot Product ≥ 0)을 갖는 단일 접선을 선택합니다.

[수식 2] 보드 하향 단일 접선 벡터 결정식 — PINBALLPHYSICSCALCULATOR.CS:83-86

$$\mathbf{T} = (\mathbf{n}_{board} \times \hat{\mathbf{n}}_{inPlane}) \,/\, ||\mathbf{n}_{board} \times \hat{\mathbf{n}}_{inPlane}||$$

$$\mathbf{T}_{down} = (\mathbf{T} \cdot \mathbf{d}_{board} \geq 0)\ ?\ +\mathbf{T} : -\mathbf{T}$$

3.3. 3중 슬라이딩 조건 판정 상태 머신 (Decision Matrix)

펭귄이 벽에 부딪혔을 때 무조건 미끄러지게 하면 고속으로 벽에 돌진했을 때 둔탁하게 달라붙는 '끈적거림(Stickiness)'이 발생합니다. 따라서 **접선 하향 이동 여부, 법선 저속 충돌 여부, 빗겨맞음(스침) 각도, 지속 충돌 여부**를 종합 평가하여 슬라이딩과 바운스를 엄격히 분기합니다.

[수식 3] 슬라이딩 vs 바운스 분기 결정 알고리즘 DECISION LOGIC

$$v_{normal} = -(v_{plane} \cdot \hat{n}_{inPlane}) \quad [\text{벽면 수직 진입 속도}]$$
$$v_{tangentDown} = v_{plane} \cdot T_{down} \quad [\text{벽면 하향 접선 속도}]$$
$$Ratio_{impact} = Clamp01(\, v_{normal} \,/\, ||v_{plane}|| \,)$$

$$isMovingDownwardOrSlow \Leftarrow (v_{tangentDown} \geq -0.2\ m/s) \vee (||v_{plane}|| \leq v_{slideThreshold})$$
$$isLowSpeed \Leftarrow (v_{normal} \leq v_{slideThreshold})$$
$$isGlancingAngle \Leftarrow (Ratio_{impact} < \cos(\theta_{glance}))$$

$$Decision\ (shouldSlide) = isMovingDownwardOrSlow \wedge (isLowSpeed \vee isGlancingAngle \vee isContinuous)$$

충돌 시나리오	판정 (Decision)	물리 거동 및 알고리즘 분기	최종 속도 벡터 산출식
저속 정면 충돌 (속력 ≤ 5.0 m/s)	SLIDE True	튕기지 않고 하향 접선 방향으로 부드럽게 유도 활주.	`(0.95·T_down + 0.05·n) · minSlideSpeed` (최소 3.5 m/s 보장)
스치는 각도 충돌 (입사각 ≥ 55°)	SLIDE True	진입 접선 속도의 85%를 보존하며 자연스럽게 하향 가속.	`(0.95·T_down + 0.05·n) · Clamp(v_tan·0.85, 3.5, 12.0)`
고속 정면 충돌 (속력 > 5.0 m/s, 직각)	BOUNCE False	벽을 따라 흐르지 않고 벽 바깥쪽으로 탄성 반사(Rebound).	`CalculateBounceVelocity(...)` (restitution: 0.25, friction: 0.15)
상향 쏘아올림 충돌 (상향 이동 중)	BOUNCE False	하향 강제 슬라이딩을 거부하고 상향 궤적을 유지하며 반사.	상향(−Z) 운동 에너지 보존 탄성 반사
벽면 지속 접촉 (OnCollisionStay)	SLIDE True	벽을 타고 내려가는 동안 지속적으로 하향 유도 가속 적용.	단, 바깥쪽 이탈 속도 감지 시 개입 즉시 중단

3.4. 벽면 파고듦 방지를 위한 5% 외측 법선 바이어스 알고리즘

💡 **95:5 바이어스 벡터 합성 메커니즘 (Penetration Prevention)**
슬라이딩 방향 벡터 산출 시 `Vector3 slideDir = (downwardTangent * 0.95f + nInPlane * 0.05f).normalized;` 수식이 적용됩니다.
순수한 100% 접선 방향(`downwardTangent`)으로만 속도를 주입하면, 수치 해석적 오차로 인해 펭귄의 구체 콜라이더가 벽면 콜라이더의 표면 법선 안쪽으로 미세하게 겹치게 됩니다. 이로 인해 유니티 PhysX의 디페네트레이션 솔버가 펭귄을 밀어내려고 강한 척력을 발생시키며 지터링(덜덜 떨림)이 유발됩니다. **5% 외측 법선 성분**은 펭귄을 벽면에서 항상 살짝 띄워주어 PhysX 간섭 없는 무저항 활주를 완성합니다.

4. PreStepVelocity 관성 보존 아키텍처 (Momentum Preservation Engine)

유니티 `MonoBehaviour`의 물리 라이프사이클에서 가장 흔히 발생하는 중대한 함정은 `OnCollisionEnter(Collision c)` 콜백 내부에서 `rigidbody.linearVelocity`를 조회하는 것입니다.

PhysX 스텝과 PreStepVelocity 캡처 시퀀스 다이어그램

```
[FixedUpdate 시작 시점]
  |
  ├──> PenguinController.FixedUpdate() : PreStepVelocity = Body.linearVelocity; [★ 충돌 전 속도 캡처]
  |
  ├──> [Unity 3D PhysX Physics Step 실행] (접점 생성, 관통 분리, 내부 반발 및 마찰 감쇄 연산 진행)
  |    ※ 이 단계에서 Body.linearVelocity는 이미 0에 가깝게 감쇄되거나 방향이 임의로 꺾임!
  |
  └──> GuideWallSurface.OnCollisionEnter(Collision c) 트리거
       ├── [기존 PhysX 조회]: Body.linearVelocity → 이미 에너지가 증발하여 둔탁하게 멈춤
       └── [PenguinShot 아키텍처]: p.PreStepVelocity 전달 → 충돌 전 순수 운동량 100% 보존 연산!
```

속도 파라미터	기존 방식 (Body.linearVelocity)	PenguinShot 아키텍처 (PreStepVelocity)
충돌 시 측정 속력	PhysX 솔버에 의해 이미 감속되어 0.1 ~ 1.5 m/s로 왜곡	충돌 직전 프레임의 실제 비행 속력 (예: 12.0 m/s) 온전히 보존
진입각 판정 신뢰도	이미 벽면에 튕겨 나온 벡터가 측정되어 스침/정면 판정 실패	벽면 진입 궤적의 순수 벡터와 법선 간의 내적이 정밀 측정됨
슬라이딩 가속 품질	충돌 후 정지 상태가 되어 최소 슬라이드 속도로만 둔탁하게 이동	스친 속도의 85%(speedRetention)가 그대로 접선 운동으로 치환

5. 표면별 물리 컴포넌트 계층 및 연계 구현 (Surface Hierarchy)

모든 반발 및 슬라이딩 표면은 `PinballBounceSurface` 추상 클래스를 상속받아 게임 상태(`WaveInProgress`)와 공통 충돌 수명주기를 완벽히 공유합니다.

GuideWallSurface (벽면 & 대기 플리퍼)

- **하단 가이드 벽**: 드레인 유입 및 플리퍼 안착 유도.
- **대기 플리퍼 표면**: `isFlipperSurface = true` 활성화 시 `flipperSlowdownFactor(0.65)` 추가 감속으로 공이 미끄러져 팁(Tip)으로 부드럽게 안착.
- **콤보/부적**: 벽 충돌 시 콤보가 누적되지 않음 (기획 준수).

FlipperController (스윙 타격 엔진)

- **스윙 시 동적 바이패스**: 플레이어가 플리퍼를 누른(스윙) 상태일 때는 `GuideWallSurface` 충돌을 건너뛰고 타격 처리.
- **하향 튕김 원천 차단**: 모서리 타격 시 법선이 하향을 가리키면 `boardUp`으로 강제 보정(`Dot < 0.2f`).
- **타격 추진력**: `18.0 m/s + incomingSpeed * 0.5f`

SlingshotBounceSurface (기획 7-2)

• **중앙 60% 반발 영역**: 충돌 위치/각도 무관하게 설치 방향(`localLaunchDirection`)으로 고정 `25.0 m/s` 부스트 발사 + 콤보 누적 + 고무줄 진동 연출.
• **양 끝 20% 비반발 영역**: 단순 벽 충돌로 취급되어 부스트 및 콤보 누적이 발생하지 않음.

Bumper & EnemyBounceSurface

• **BumperBounceSurface**: 진입 속도에 반발 배율(1.15배) 및 최소 속도(6.0 m/s)를 보장하여 역동적 핀볼 반사 유도.
• **EnemyBounceSurface**: 적 충돌 시 데미지 처리 및 넉백 부여. 펭귄 과부화(Overload) 상태 진입 시 고스트 레이어로 관통 지원.

6. 제로 마찰/제로 반발 물리 재질(Physics Material) SSOT 구성

유니티 씬의 비결정론적 마찰 오차를 제거하고 C# 도메인 물리 계산기의 완전한 제어권을 보장하기 위해, 게임 내 핵심 강체 재질은 모두 **완전 무마찰·무반발(Zero Friction & Restitution)**로 설정되어 있습니다.

에셋 파일명 (PhysicMaterial)	Dynamic Friction	Static Friction	Bounciness	Combine Modes	역할 및 설계 목적
`PenguinMaterial.physicMaterial`	**0.0**	**0.0**	**0.0**	Friction: Minimum Bounce: Average	펭귄 몸체의 기계적 걸림을 배제하고 순수 스탯(MaxSpeed, LinearDrag)에 의한 감쇄만 허용.
`BoardMaterial.physicMaterial`	**0.0**	**0.0**	**0.0**	Friction: Minimum Bounce: Average	바닥판 및 가이드 벽면의 기본 저항을 제로화하여 `PinballPhysicsCalculator` 연산이 왜곡 없이 반영됨.
`FilpperMaterial.physicMaterial`	0.6	0.6	0.0	Friction: Average Bounce: Minimum	플리퍼 스윙 타격 시 HingeJoint 접촉 면에서 공이 헛돌지 않고 마찰력을 받아 회전 토크를 추진력으로 전환.

7. 펭귄 끼임 방지(Anti-Stuck) 및 드레인 수명주기 관리

복잡한 핀볼 맵 구조 상 펭귄이 다수의 범퍼 사이에 끼이거나 기하학적 결함으로 정지하는 상황에 대비한 3단계 안전장치가 상시 가동됩니다.

펭귄 끼임 감지(CheckStuckCondition) 및 수명주기 전이 파이프라인

1단계: 발사 유예 기간 (Launch Grace Period)
• 펭귄 발사 직후 `launchGracePeriod (1.0초)` 동안은 끼임 검사를 엄격히 유예하여 초기 슬링샷 조준/발사 구간 오작동 차단.

2단계: 주기적 이동 변위 검사 (Displacement Check)
• `stuckCheckInterval (1.0초)` 주기마다 이전 검사 위치와의 변위 `distanceMoved = Distance(pos, lastCheckedPos)` 측정.
• 이동 변위가 `minMoveThreshold (0.1m)` 미만일 경우 정체(Stuck) 상태로 즉시 확정.

3단계: 자동 드레인(Drain) 및 부화장 리스폰 (Safe Evacuation)
• `Drain()` 메서드 호출 → 필드 비활성화, 콤보 리셋, `GameEvents.RaisePenguinDrained` 이벤트 발행.
• 펭귄의 `RespawnTime` 스탯 카운트다운 후 안전하게 부화장 슬롯으로 복귀(CompleteRespawn).

8. 단위 테스트(EditMode) 검증 및 성능 평가 (Verification & Performance)

본 시스템의 핵심 도메인 로직은 씬 로드 및 렌더링 오버헤드가 전무한 순수 C# EditMode 단위 테스트 프레임워크를 통해 철저하게 검증되었습니다.

테스트 케이스명	검증 시나리오 및 입력값 조건	기대 물리 결과	검증 상태
`CalculateSlideVelocity_LowSpeedHeadOnImpact_SlidesDownSmoothly`	시속 3.0 m/s로 벽면에 정면 충돌 (X=3.0, Z=0.0). 진입각 임계치 55도, 슬라이드 임계 속도 5.0 m/s.	반사되지 않고 +Z(하향) 슬라이딩 속도 발생 및 최소 슬라이드 속도(3.5 m/s) 이상 유지.	PASS
`CalculateSlideVelocity_GlancingAngle_FollowsDownwardTangent`	얕은 각도로 비스듬히 벽면 진입 (X=1.0, Z=8.0, 빗겨맞음).	+Z 하향 방향으로 미끄러지며, 진입 속도에 보존율(85%)이 정확히 곱해진 속력 산출.	PASS
`CalculateSlideVelocity_HighSpeedHeadOnImpact_BouncesAwayFromWall`	시속 10.0 m/s의 고속으로 벽면에 정면 수직 충돌 (X=10.0, Z=0.0).	하향으로 쏠리지 않고(Z=0), 벽 법선 바깥쪽(-X)으로 강하게 탄성 튕김.	PASS
`CalculateSlideVelocity_UpwardMovingPenguin_DoesNotForceDownwardSlide`	플리퍼 타격 후 상향 비행 중 벽면 충돌 (X=3.0, Z=-7.0, 상향 이동).	하향 강제 슬라이딩을 거부하고, 계속 상향(-Z) 성분을 유지하며 벽 바깥으로 반사.	PASS
`Slingshot_CenterPoint_IsRepulsionZone`	슬링샷 막대 중앙 좌표(0, 0, 0) 충돌 판정.	중앙 60% 반발 영역 (IsRepulsionZone = True) 확정.	PASS
`Slingshot_OutsideRatioThreshold_IsNonRepulsionZone`	막대 중심 기준 1.5m, 1.8m 지점 (양 끝 20% 외곽) 충돌 판정.	비반발 영역 (IsRepulsionZone = False)으로 단순 벽 충돌 판정.	PASS

8.2. GC(Garbage Collection) 및 핫패스(Hot-Path) 성능 프로파일링

> ⚡ **Zero-Allocation Runtime Performance**
> `FixedUpdate`, `OnCollisionEnter`, `OnCollisionStay` 등 매 프레임 수십 회 이상 호출되는 핫패스 내에서 동적 힙 할당(`new` 키워드), LINQ 쿼리 호출, 박싱/언박싱이 100% 배제되었습니다. 모든 벡터 연산은 스택 메모리(Struct Value Type) 상에서 인라인으로 처리되며, GC 프레셔 발생률은 **0.00 KB/Frame**을 달성하였습니다.

9. 결론 및 향후 고도화 로드맵 (Conclusion & Roadmap)

본 펭귄 물리 시스템은 유니티 상용 물리 엔진(PhysX)의 구조적 한계인 마찰 멈춤 현상과 충돌 시점의 속도 왜곡 문제를 **PreStepVelocity 캐싱**과 **수학적 벡터 정사영 기반의 GuideWall 유도 알고리즘**을 통해 완벽하게 해결하였습니다.

결과적으로 플레이어는 펭귄을 조준 발사했을 때 시원한 타격감과 탄성 튕김을 만끽함과 동시에, 보드 하단 벽면 및 플리퍼 주변에서 공이 끼이거나 역주행하지 않고 플리퍼 타격존으로 매끄럽게 흘러내려오는 최적의 핀볼 손맛과 조작 안정성을 경험할 수 있습니다.

> 🚀 **차기 고도화 권장 과제 (Roadmap)**
> 1. **Catmull-Rom 곡면 가이드 벽면 대응**: 현재 평면/직선 박스 콜라이더 기반 유도에서, 스플라인 곡선 벽면을 따라 연속 미분 접선을 추적하는 곡면 슬라이딩 확장.
> 2. **동적 보드 경사각(Pitch Tilt) 물리 보정**: 향후 스테이지 기믹으로 보드가 좌우/상하로 기울어질 때 `boardDown` 벡터를 실시간 동적 갱신하는 틸트 시스템 연동.